

MYSQL ASSIGNMENT 2

CLAUSES AND JOINS

PREPARED BY:

AMRA ABDUL HAMEED

The Database employee is used here, from the previous assignment. The table Employee Data is populated using the given DML commands.

```
USE employee;
```

```
INSERT INTO departments (department_id, department_name) VALUES
(1, 'Software Development'),
(2, 'Marketing'),
(3, 'Data Science'),
(4, 'Human Resources'),
(5, 'Product Management'),
(6, 'Content Creation'),
(7, 'Finance'),
(8, 'Design'),
(9, 'Research and Development'),
(10, 'Customer Support'),
(11, 'Business Development'),
(12, 'IT'),
(13, 'Operations');
```

```
INSERT INTO location (location) VALUES
('Chennai'),
('Bangalore'),
('Hyderabad'),
('Pune');
```

```
INSERT INTO employees (employee_id, employee_name, gender, age, hire_date,
designation, department_id, location_id, salary) VALUES
(5001, 'Vihaan Singh', 'M', 27, '2015-01-20', 'Data Analyst', 3, 4,
60000),
(5002, 'Reyansh Singh', 'M', 31, '2015-03-10', 'Network Engineer', 12, 1,
80000),
(5003, 'Aaradhya Iyer', 'F', 26, '2015-05-20', 'Customer Support
Executive', 10, 2, 45000),
(5004, 'Kiara Malhotra', 'F', 29, '2015-07-05', NULL, 8, 3, 70000),
(5005, 'Anvi Chaudhary', 'F', 25, '2015-09-11', 'Business Development
Executive', 11, 1, 55000),
(5006, 'Dhruv Shetty', 'M', 28, '2015-11-20', 'UI Developer', 8, 2,
65000),
(5007, 'Anushka Singh', 'F', 32, '2016-01-15', 'Marketing Manager', 2, 3,
90000),
(5008, 'Diya Jha', 'F', 27, '2016-03-05', 'Graphic Designer', 8, 4,
70000),
(5009, 'Kiaan Desai', 'M', 30, '2016-05-20', 'Sales Executive', 11, 3,
55000),
(5010, 'Atharv Yadav', 'M', 29, '2016-07-10', 'Systems Administrator', 12,
4, 80000),
(5011, 'Saanvi Patel', 'F', 28, '2016-09-20', 'Marketing Analyst', 2, 1,
60000),
(5012, 'Myra Verma', 'F', 26, '2016-11-05', 'Operations Manager', 13, 2,
95000),
(5013, 'Arnav Rao', 'M', 33, '2017-01-20', 'Customer Success Manager', 10,
3, 75000),
```

```

(5014, 'Vihaan Mohan', 'M', 30, '2017-03-10', 'Supply Chain Analyst', 10, 2, 60000),
(5015, 'Ishaan Kumar', 'M', 27, '2017-05-20', 'Financial Analyst', 7, 1, 85000),
(5016, 'Zoya Khan', 'F', 31, '2017-07-05', 'Legal Counsel', 4, 4, 100000),
(5017, 'Kabir Nair', 'M', 28, '2017-09-11', 'IT Support Specialist', 12, 2, 80000),
(5018, 'Ishan Mishra', 'M', 25, '2017-11-20', 'Research Scientist', 9, 3, 75000),
(5019, 'Ishika Patel', 'F', 29, '2018-01-15', 'Talent Acquisition Specialist', 4, 4, 55000),
(5020, 'Aarav Nair', 'M', 32, '2018-03-05', 'Software Engineer', 1, 1, 90000),
(5021, 'Advik Kapoor', 'M', 26, '2018-05-20', 'Finance Analyst', 7, 3, 85000),
(5022, 'Aadhya Iyengar', 'F', 28, '2018-07-10', 'HR Specialist', 4, 4, 60000),
(5023, 'Anika Paul', 'F', 30, '2018-09-20', 'Public Relations Specialist', 2, 2, 70000),
(5024, 'Aryan Shetty', 'M', 27, '2018-11-05', 'Product Manager', 5, 1, 95000),
(5025, 'Avni Iyengar', 'F', 31, '2019-01-20', 'Data Scientist', 3, 4, 100000),
(5026, 'Vivaan Singh', 'M', 29, '2019-03-10', 'Business Analyst', 3, 2, 75000),
(5027, 'Ananya Paul', 'F', 32, '2019-05-20', 'Content Writer', 6, 3, 60000),
(5028, 'Anaya Kapoor', 'F', 26, '2019-07-05', 'Event Coordinator', 6, 1, 60000),
(5029, 'Arjun Kumar', 'M', 33, '2019-09-11', 'Quality Assurance Analyst', 12, 2, 80000),
(5030, 'Sara Iyer', 'F', 28, '2019-11-20', 'Project Manager', 5, 1, 90000);

```

57 • **SELECT * FROM DEPARTMENTS;**

Result Grid   Filter Rows: Edit:   

	department_id	department_name
▶	11	Business Development
	6	Content Creation
	10	Customer Support
	3	Data Science
	8	Design
	7	Finance
	4	Human Resources
	12	IT
	2	Marketing
	13	Operations

mysqlassgnmt1script* SQL File 3*

Limit to 1000 rows

```

49 (5024, 'Aryan Shetty', 'M', 27, '2018-11-05', 'Product Manager', 5, 1, 95000),
50 (5025, 'Avni Iyengar', 'F', 31, '2019-01-20', 'Data Scientist', 3, 4, 100000),
51 (5026, 'Vivaan Singh', 'M', 29, '2019-03-10', 'Business Analyst', 3, 2, 75000),
52 (5027, 'Ananya Paul', 'F', 32, '2019-05-20', 'Content Writer', 6, 3, 60000),
53 (5028, 'Anaya Kapoor', 'F', 26, '2019-07-05', 'Event Coordinator', 6, 1, 60000),
54 (5029, 'Arjun Kumar', 'M', 33, '2019-09-11', 'Quality Assurance Analyst', 12, 2, 80000),
55 (5030, 'Sara Iyer', 'F', 28, '2019-11-20', 'Project Manager', 5, 1, 90000);
56
57 • SELECT * FROM Employees;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

employee_id	employee_name	gender	age	hire_date	department_id	location_id	salary	designation
5001	Vihaan Singh	M	27	2015-01-20	3	4	60000.00	Data Analyst
5002	Reyansh Singh	M	31	2015-03-10	12	1	80000.00	Network Engineer
5003	Aaradhya Iyer	F	26	2015-05-20	10	2	45000.00	Customer Support Executive
5004	Kiara Malhotra	F	29	2015-07-05	8	3	70000.00	UI Developer
5005	Anvi Chaudhary	F	25	2015-09-11	11	1	55000.00	Business Development Executive
5006	Dhruv Shetty	M	28	2015-11-20	8	2	65000.00	Marketing Manager
5007	Anushka Singh	F	32	2016-01-15	2	3	90000.00	Graphic Designer
5008	Diya Jha	F	27	2016-03-05	8	4	70000.00	Sales Executive
5009	Kiaan Desai	M	30	2016-05-20	11	3	55000.00	

Employees 1 x

Output: Action Output

```

58 • SELECT * FROM LOCATION;

```

Result Grid | Filter Rows: |

location_id	location_name
2	Bangalore
1	Chennai
3	Hyderabad
4	Pune
NULL	NULL

Execute the queries in this assignment after inserting the data.

1. Distinct Values:

● a query to retrieve distinct salaries from the Employees table.

SELECT DISTINCT salary FROM employees;

```

61 • SELECT DISTINCT salary
62 FROM employees;

```

salary
60000.00
80000.00
45000.00
70000.00
55000.00
65000.00
90000.00
95000.00
75000.00
55000.00

2. Alias (AS):

☉ Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_Salary", respectively.

SELECT

age AS Employee_Age,

salary AS Employee_Salary

FROM employees;

```

4 • SELECT
5     age AS Employee_Age,
6     salary AS Employee_Salary
7 FROM employees;

```

Employee_Age	Employee_Salary
27	60000.00
31	80000.00
26	45000.00
29	70000.00
25	55000.00
28	65000.00
32	90000.00
27	70000.00
30	55000.00
28	65000.00

3. Where Clause & Operators:

● Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.

```
SELECT *
```

```
FROM employees
```

```
WHERE salary > 50000
```

```
AND hire_date < '2016-01-01';
```

```
69 • SELECT *
70 FROM employees
71 WHERE salary > 50000
72 AND hire_date < '2016-01-01';
```

	employee_id	employee_name	gender	age	hire_date	department_id	location_id	salary	designation
▶	5001	Vihaan Singh	M	27	2015-01-20	3	4	60000.00	Data Analyst
	5002	Reyansh Singh	M	31	2015-03-10	12	1	80000.00	Network Engineer
	5004	Kiara Malhotra	F	29	2015-07-05	8	3	70000.00	NULL
	5005	Anvi Chaudhary	F	25	2015-09-11	11	1	55000.00	Business Development Executive
	5006	Dhruv Shetty	M	28	2015-11-20	8	2	65000.00	UI Developer
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

● Find the employee whose designation is missing and fill it with "Data Scientist".

```
74 • SELECT *
75 FROM employees
76 WHERE designation IS NULL;
```

	employee_id	employee_name	gender	age	hire_date	department_id	location_id	salary	designation
▶	5004	Kiara Malhotra	F	29	2015-07-05	8	3	70000.00	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
UPDATE employees
SET designation = 'Data Scientist'
WHERE employee_id = 5004;
```

13

```
74 • SELECT *
75 FROM employees
76 WHERE employee_id = 5004;
```

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
	employee_id	employee_name	gender	age	hire_date	department_id	location_id	salary	designation
▶	5004	Kiara Malhotra	F	29	2015-07-05	8	3	70000.00	Data Scientist
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Sorting and Grouping Data:

1. ORDER BY:

🕒 Find employees sorted by department ID in ascending order and salary in descending order.

```
82 • SELECT *
83 FROM employees
84 ORDER BY department_id ASC, salary DESC;
```

Result Grid		Filter Rows:		Edit:			Export/Import:			Wrap Cell Content:	
	employee_id	employee_name	gender	age	hire_date	department_id	location_id	salary	designation		
▶	5020	Aarav Nair	M	32	2018-03-05	1	1	90000.00	Software Engineer		
	5007	Anushka Singh	F	32	2016-01-15	2	3	90000.00	Marketing Manager		
	5023	Anika Paul	F	30	2018-09-20	2	2	70000.00	Public Relations Specialist		
	5011	Saanvi Patel	F	28	2016-09-20	2	1	60000.00	Marketing Analyst		
	5025	Avni Iyengar	F	31	2019-01-20	3	4	100000.00	Data Scientist		
	5026	Vivaan Singh	M	29	2019-03-10	3	2	75000.00	Business Analyst		
	5001	Vihaan Singh	M	27	2015-01-20	3	4	60000.00	Data Analyst		
	5016	Zoya Khan	F	31	2017-07-05	4	4	100000.00	Legal Counsel		
	5022	Aadhya Iyengar	F	28	2018-07-10	4	4	60000.00	HR Specialist		
	5010	Aditya Iyengar	F	28	2018-01-15	1	1	55000.00	Product Development Specialist		

employees 9 x

Output

2. LIMIT:

🕒 Display the first 5 employees hired in the year 2018.

SELECT *

FROM employees

WHERE hire_date >= '2018-01-01'

AND hire_date < '2019-01-01'

ORDER BY hire_date

LIMIT 5;

```
86 • SELECT *
87 FROM employees
88 WHERE hire_date >= '2018-01-01'
89 AND hire_date < '2019-01-01'
90 ORDER BY hire_date
91 LIMIT 5;
```

	employee_id	employee_name	gender	age	hire_date	department_id	location_id	salary	designation
▶	5019	Ishika Patel	F	29	2018-01-15	4	4	55000.00	Talent Acquisition Specialist
	5020	Aarav Nair	M	32	2018-03-05	1	1	90000.00	Software Engineer
	5021	Advik Kapoor	M	26	2018-05-20	7	3	85000.00	Finance Analyst
	5022	Aadhya Iyengar	F	28	2018-07-10	4	4	60000.00	HR Specialist
	5023	Anika Paul	F	30	2018-09-20	2	2	70000.00	Public Relations Specialist
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3. Aggregate Functions:

- Calculate the sum of all salaries in the Finance department.

SELECT SUM(salary) AS Total_Finance_Salary

FROM employees

WHERE department_id = 7;

```
93 • SELECT SUM(salary) AS Total_Finance_Salary
94 FROM employees
95 WHERE department_id = 7;
```

	Total_Finance_Salary
▶	170000.00

- Find the minimum age among all employees.

SELECT MIN(age) AS Minimum_Age

FROM employees;

```
96
97 • SELECT MIN(age) AS Minimum_Age
98 FROM employees;
```

Result Grid | Filter Rows: | Export:

Minimum_Age
25

4. GROUP BY:

- List the maximum salary for each location.

SELECT

l.location_name,

MAX(e.salary) AS Maximum_Salary

FROM employees e

JOIN location l

ON e.location_id = l.location_id

GROUP BY l.location_name;

```
100 • SELECT
101     l.location_name,
102     MAX(e.salary) AS Maximum_Salary
103 FROM employees e
104 JOIN location l
105     ON e.location_id = l.location_id
106 GROUP BY l.location_name;
107
108
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

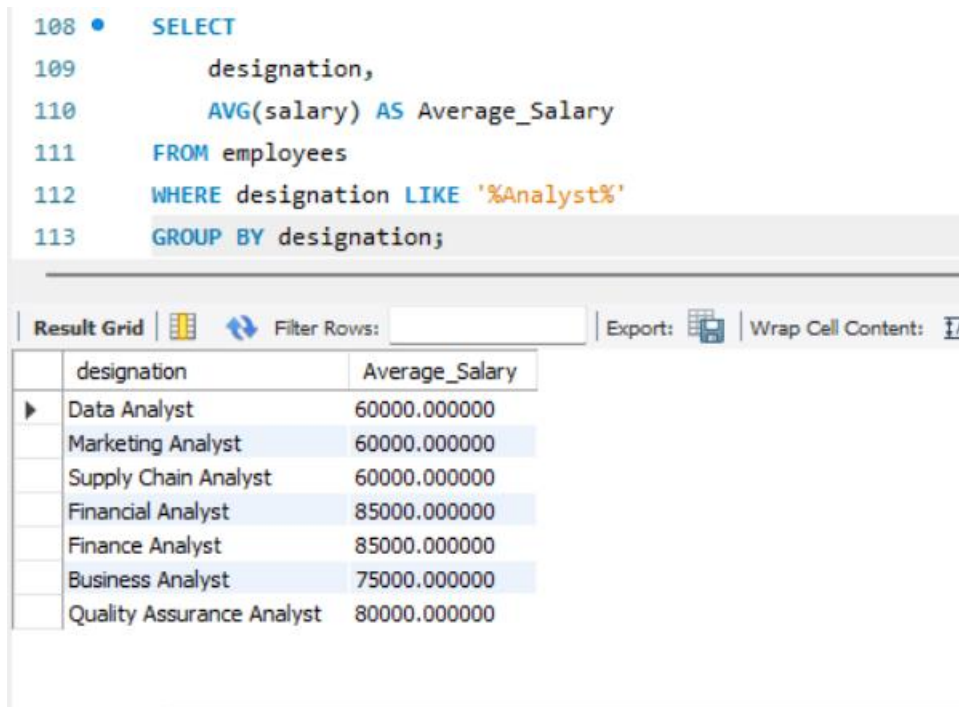
location_name	Maximum_Salary
Bangalore	95000.00
Chennai	95000.00
Hyderabad	90000.00
Pune	100000.00

- Calculate the average salary for each designation containing the word 'Analyst'.

```

SELECT
    designation,
    AVG(salary) AS Average_Salary
FROM employees
WHERE designation LIKE '%Analyst%'
GROUP BY designation;

```



108 • SELECT
 109 designation,
 110 AVG(salary) AS Average_Salary
 111 FROM employees
 112 WHERE designation LIKE '%Analyst%'
 113 GROUP BY designation;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	designation	Average_Salary
▶	Data Analyst	60000.000000
	Marketing Analyst	60000.000000
	Supply Chain Analyst	60000.000000
	Financial Analyst	85000.000000
	Finance Analyst	85000.000000
	Business Analyst	75000.000000
	Quality Assurance Analyst	80000.000000

5. HAVING:

🔴 Find departments with less than 3 employees.

```

SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS Employee_Count
FROM departments d
LEFT JOIN employees e
    ON d.department_id = e.department_id

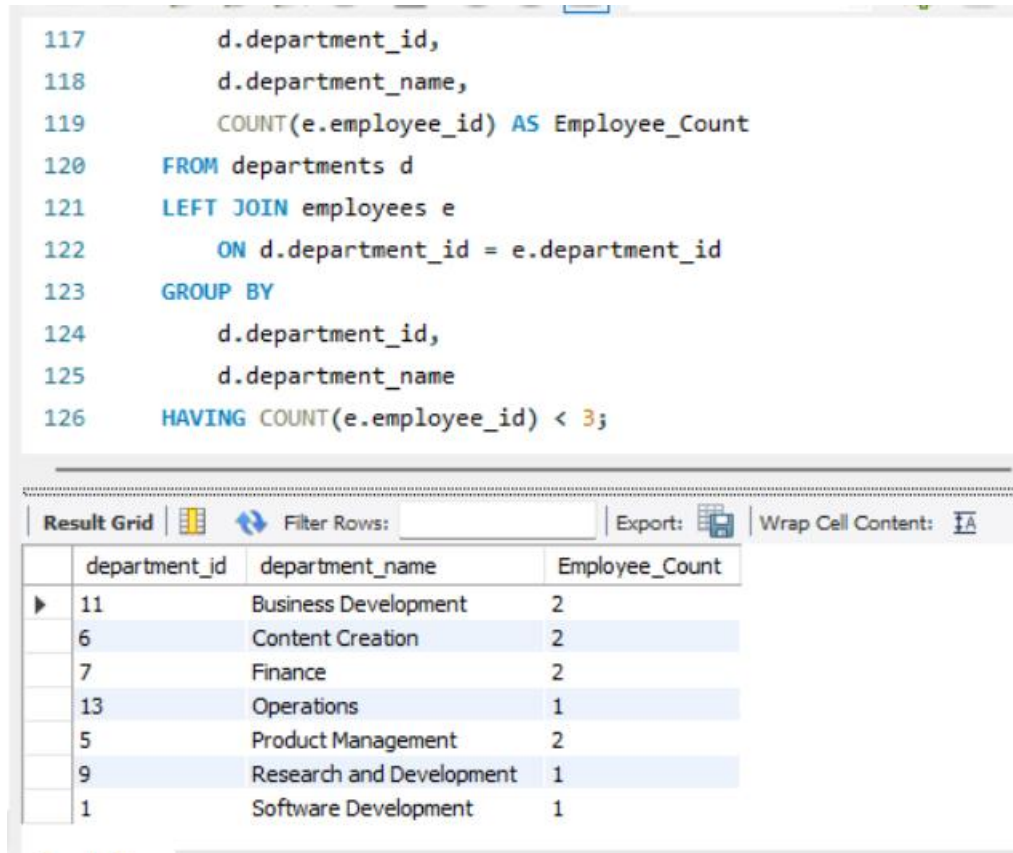
```

GROUP BY

d.department_id,

d.department_name

HAVING COUNT(e.employee_id) < 3;



The screenshot shows a SQL query editor with a query and its results in a grid. The query is as follows:

```
117     d.department_id,  
118     d.department_name,  
119     COUNT(e.employee_id) AS Employee_Count  
120 FROM departments d  
121 LEFT JOIN employees e  
122     ON d.department_id = e.department_id  
123 GROUP BY  
124     d.department_id,  
125     d.department_name  
126 HAVING COUNT(e.employee_id) < 3;
```

The results are displayed in a grid with the following columns: department_id, department_name, and Employee_Count.

department_id	department_name	Employee_Count
11	Business Development	2
6	Content Creation	2
7	Finance	2
13	Operations	1
5	Product Management	2
9	Research and Development	1
1	Software Development	1

🔍 Find locations with female employees whose average age is below 30.

SELECT

l.location_name,

AVG(e.age) AS Average_Female_Age

FROM employees e

JOIN location l

ON e.location_id = l.location_id

WHERE e.gender = 'F'

GROUP BY l.location_name

HAVING AVG(e.age) < 30;

```

128 • SELECT
129     l.location_name,
130     AVG(e.age) AS Average_Female_Age
131 FROM employees e
132 JOIN location l
133     ON e.location_id = l.location_id
134 WHERE e.gender = 'F'
135 GROUP BY l.location_name
136 HAVING AVG(e.age) < 30;

```

Result Grid			Filter Rows:	Export:	Wrap
	location_name	Average_Female_Age			
▶	Bangalore	27.3333			
	Chennai	26.7500			
	Pune	29.2000			

Joins:

1. Inner Join:

● List employee names, their designations, and department names where employees are assigned to a department.

SELECT

```

    e.employee_name,
    e.designation,
    d.department_name

```

FROM employees e

INNER JOIN departments d

```

    ON e.department_id = d.department_id;

```

```

138 • SELECT
139     e.employee_name,
140     e.designation,
141     d.department_name
142 FROM employees e
143 INNER JOIN departments d
144     ON e.department_id = d.department_id;

```

Result Grid			
Filter Rows:			
Export:			
Wrap Cell Content:			
	employee_name	designation	department_name
▶	Anvi Chaudhary	Business Development Executive	Business Development
	Kiaan Desai	Sales Executive	Business Development
	Ananya Paul	Content Writer	Content Creation
	Anaya Kapoor	Event Coordinator	Content Creation
	Aaradhya Iyer	Customer Support Executive	Customer Support
	Arnav Rao	Customer Success Manager	Customer Support
	Vihaan Mohan	Supply Chain Analyst	Customer Support
	Vihaan Singh	Data Analyst	Data Science

Result 19 x

2. Left Join:

- List all departments along with the total number of employees in each department, including departments with no employees.

```

SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS Total_Employees
FROM departments d
LEFT JOIN employees e
    ON d.department_id = e.department_id
GROUP BY
    d.department_id,
    d.department_name;

```

```

146 • SELECT
147     d.department_id,
148     d.department_name,
149     COUNT(e.employee_id) AS Total_Employees
150 FROM departments d
151 LEFT JOIN employees e
152     ON d.department_id = e.department_id
153 GROUP BY
154     d.department_id,
155     d.department_name;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
department_id	department_name	Total_Employees	
11	Business Development	2	
6	Content Creation	2	
10	Customer Support	3	
3	Data Science	3	
8	Design	3	
7	Finance	2	
4	Human Resources	3	
12	IT	4	

3. Right Join:

☉ Display all locations along with the names of employees assigned to each location. If no employees are assigned to a location, display NULL for employee name.

SELECT

l.location_name,
e.employee_name

FROM employees e

RIGHT JOIN location l

ON e.location_id = l.location_id;

```

157 • SELECT
158     l.location_name,
159     e.employee_name
160 FROM employees e
161 RIGHT JOIN location l
162     ON e.location_id = l.location_id;

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	location_name	employee_name
▶	Bangalore	Aaradhya Iyer
	Bangalore	Dhruv Shetty
	Bangalore	Myra Verma
	Bangalore	Vihaan Mohan
	Bangalore	Kabir Nair
	Bangalore	Anika Paul
	Bangalore	Vivaan Singh
	Bangalore	Ariun Kumar

Result 21 ×

Output